

Project 1: Software Product from Idea to Execution

Jovany Cardenas, Gagandeep Singh

Department of Engineering, San José State University

CMPE 165: Project 1

Professor Borthakur

September 2026

Part A – Mission and Strategic Alignment

A1. Mission

Our mission is to provide a simple and accessible way for individuals to keep track of their everyday expenses and to better understand where their money is being spent.

A2. Problem

Many people in everyday life have difficulty keeping track of their daily spending, especially when purchases are spread across different categories and happen throughout the week or month. Furthermore, small expenses can easily add up without people realizing it. Our expense tracker is intended to give users one place to record and review all their spending so they can have a clear view of their financial habits.

A3. Project Objective

The main objective of the project is to create a simple-to-use Expense Tracker that allows users to record expenses and review their spending information in an organized way. The application is intended to make basic expense tracking easier without requiring complicated financial software.

A4. Strategic Alignment

The Expense Tracker supports the organization's goal of providing simple financial tools that help users better manage their money. The project could improve the user experience by making spending easier to track and understand while also providing opportunities for future growth.

A5. Success Metrics

The project will be considered successful if:

- At least 95% of automated tests pass.
- All core features are working by the project deadline.
- Users can add and review an expense in less than one minute.
- Users cannot access another user's financial information.

Part B – Stakeholders, Organization, and Culture

B1. Stakeholders

Stakeholder	Wants	Influence	Interest	Position
Users	Simple and private expense tracking	High	High	Support
Developers	Clear requirements and realistic scope	High	High	Support
Project Manager	On-time completion	High	High	Support
Business Owner / Investor	Useful and financially viable product	High	Medium	Support if financially viable
IT Operations	Reliable and manageable infrastructure	Medium	Medium	Support

Power-Interest Matrix

	Low Interest	High Interest
High Power	Business Owner / Investor	Project Manager, Development Team, Users
Low Power	Hosting / IT Operations	

B2. Organizational Structure

A projectized organizational structure would be appropriate for the Expense Tracker project because the team is focused mainly on completing one specific product. Team members can work directly toward the same goal without having to report through several separate

departments. The project manager would have clear responsibility for coordinating the schedule, priorities, and major project decisions.

One advantage of this structure is that it creates clear ownership and can allow decisions to be made faster. Since the team is organized around the Expense Tracker, it is easier to keep everyone focused on the same product goals.

One disadvantage is that expertise may be duplicated or limited. A small project team may not have dedicated specialists for areas such as security, testing, or user-interface design, so team members may need to take on multiple roles.

B3. Organizational Culture

Three characteristics that would help the project succeed are collaboration, open communication, and willingness to improve based on feedback. These would help the team identify problems and work together to solve them. A culture of blaming people for mistakes could hurt the project because team members may become less willing to report problems or ask for help.

Part C – Project Selection

The Expense Tracker was compared with an Inventory Management System using a weighted scoring model. Each project was scored from 1 to 5, with 5 being the strongest.

Criteria	Weight	Expense Tracker	Weighted	Inventory Management	Weighted
User Value	25%	5	1.25	5	1.25
Feasibility	25%	5	1.25	4	1.0
Technical Learning	20%	4	0.8	4	0.8
Future Growth	15%	5	0.75	5	0.75
Implementation Risk	15%	4	0.6	3	0.45
Total	100%		4.65		4.25

Based on the weighted scoring model, the Expense Tracker received a score of 4.65 compared with 4.25 for the Inventory Management System. The Expense Tracker was selected because it scored higher overall and was considered easier to complete within the two-week project period.

Part D – Financial Analysis

For this analysis, the Expense Tracker is predicted to have an initial development cost of \$25,000. This includes development time, testing, setup, and other expenses related to the overall project. The Project is also expected to create value over 4 years through subscription revenue and premium features. Additionally, savings from the use of automation and reduced support could be realized as well.

The Estimated Yearly Benefits:

Year	Estimated Benefit
1	\$8,000
2	\$10,000
3	\$12,000
4	\$14,000

10% Discount Rate:

Year	Cash Flow	Present Value
1	\$8,000	\$7,272.73
2	\$10,000	\$8,264.46
3	\$12,000	\$9,015.78
4	\$14,000	\$9,561.16

Total Present Value:

$$\$7,272.73 + \$8,264.46 + \$9,015.78 + \$9,561.16 = \$34,114.13$$

NPV Value:

$$\$34,114.13 - \$25,000 = \$9,114.13$$

$$\text{NPV} = \$9,114.13$$

Management should fund this project based on the NPV value, having a positive NPV of \$9,114.13. This means that after the future benefits are discounted back to their present value, the project is still expected to generate more financial value than the original \$25,000 investment. This is because the project would not just earn back its initial costs, but instead it is expected to create additional value. Moreover, a positive NPV also suggests that the estimated benefits are strong enough to justify the investment. Therefore, from a financial point of view, it would be worth pursuing this project. The main limitation is that the results depend on the estimates being accurate.

Part E – Leadership and Ethics

E1. Leadership

A collaborative leadership style would work well for the Expense Tracker project. Since the project is being completed by a small team, everyone can have some input when decisions are made. Furthermore, the project manager should keep everyone in the team organized, make sure work is being done on time, and be able to step in whenever there are disagreements or confusion.

Team members should also have enough freedom to make decisions about the part of the project they are working on. Moreover, not every small decision needs approval from the project manager. However, large decisions that affect the entire application, such as removing an important feature or changing the project scope, should be discussed by the whole team.

If the project falls behind schedule, the project manager should first figure out what is causing the delay. The team could then change priorities, reduce less important features, or move work between members instead of simply rushing everything at the end.

E2. Leadership Antipattern

One leadership problem that could hurt the project is micromanagement. For example, if the project manager tries to control every small decision, team members may spend too much time waiting for approval instead of completing their work. In addition, it could also make

communication worse because team members may become less willing to suggest their own ideas or point out problems.

E3. Ethical Challenge

One ethical issue that could occur is the exposure of users' personal financial information. This could affect users, the development team, and the organization operating the application. The project manager should make privacy a priority by limiting access to financial information, testing that users cannot access another user's records, and fixing any privacy problems before adding less important features.

Part F – Team Design

A professional team building the full version of the Expense Tracker would include a project manager, software developers, a product or requirements lead, and someone responsible for testing and quality.

Technical decisions would mainly be made by the developers, especially decisions involving the design of the application and how features are implemented. Product requirements would be decided by the product lead based on the needs of the users. The project manager would be responsible for tracking the schedule and making sure the team stays on task. Quality would be shared by the developers and testers rather than being the responsibility of only one person. If disagreements happen, the team should first discuss the issue and compare the advantages and disadvantages of each option. For larger decisions, the project manager can help the group reach a decision if the team cannot agree.

A hybrid team could work well for this project. Team members could meet in person when major decisions, planning, or testing need to be done together, while still completing individual development work remotely. This gives the team flexibility without completely losing face-to-face communication.

Part G – Conflict and Negotiation

One disagreement that could happen during the Expense Tracker project is whether the team should continue adding features or focus on finishing and improving the existing features. One side may want to add features such as savings goals, recurring transactions, receipt uploads, or bank account integration because these could make the application more useful. The other side may want to keep the scope smaller so the team has enough time to test the application and make sure the main features work correctly.

The conflict exists because both sides want the project to succeed, but they have different ideas about what is more important. One side values having more functionality while the other side values reliability and completing the project on time.

The project manager should have the team identify which features are actually necessary for the main purpose of the Expense Tracker. A possible compromise would be to complete the important features first and move the extra features into a future version of the application. This would allow the team to finish a reliable prototype while still keeping the additional ideas for later development.

Part H – Risk Identification

The Expense Tracker project has several risks that could affect the quality of the application or the team's ability to complete it on time. The risks were ranked using a probability score and an impact score from 1 to 5. The overall risk score is then calculated by multiplying the probability by the impact.

Risk Score = Probability x Impact

Risk	Category	Probability	Impact	Risk Score	Risk Owner
Schedule delays	Schedule	4	4	16	Project Manager
Integration or configuration problems	Technical	4	4	16	Development Team
Security or Privacy issues	Security	3	5	15	Development Team
Incorrect calculations or financial data	Quality/ Technical	3	4	12	Developer/ Testers

Risk	Category	Probability	Impact	Risk Score	Risk Owner
Scope Creep from adding too many features	Scope	4	3	12	Project Manager
Low user adoption or poor usability	Business/ User	3	3	9	Product Lead

The three highest priority risks are schedule delays, integration or configuration problems, and security or privacy issues. Schedule delays are a major concern because the project has a short development period where testing and debugging can take longer than expected. Integration and configuration problems are also a high risk because the team had to adapt existing Django code and make different parts of the application work together. Security and privacy are important as well because the Expense Tracker stores personal financial information that could potentially get leaked, and this could have a serious impact even if its probability is lower than some of the other risks.

Part I – Decision Tree

One management decision for the Expense Tracker is whether the team should launch the application after the normal testing process or spend additional time on extra testing and security review.

Option A - Launch After Normal Testing

If the team launches after the normal testing process:

- 75% chance of a successful launch with a financial value of \$18,000
- 25% chance of a major issue after launch with a financial impact of -\$8,000

Expected Monetary Value:

$$EMV = (0.75 \times \$18,000) + (0.25 \times -\$8,000)$$

$$\text{EMV} = \$13,500 - \$2,000$$

$$\text{EMV} = \$11,500$$

Option B - Additional Testing and Security Review

If the team spends more time on testing and security review:

- 90% chance of a successful launch with a financial value of \$14,000
- 10% chance of a major issue with a financial impact of -\$4,000

Expected Monetary Value:

$$\text{EMV} = (0.90 \times \$14,000) + (0.10 \times \$4,000)$$

$$\text{EMV} = \$12,600 - \$400$$

$$\text{EMV} = \$12,200$$

Based on the decision tree, management should choose option B and perform the additional testing and security review. With option B, there is a higher expected monetary value of \$12,000 compared to \$11,500 for option A. The difference may not seem very large; however, the additional testing also lowers the chance of a serious issue after launch.

Part J – Monte Carlo Simulation

For the Monte Carlo simulation, the uncertain outcome selected was the total development cost of the Expense Tracker. As of now, the project is estimated to cost about \$25,000, but the final cost could change depending on the development time, testing and debugging, and other setup expenses.

Three uncertain variables were used in the simulation:

Variable	Minimum	Most Likely	Maximum
Development labor	\$15,000	\$18,000	\$22,000
Testing and debugging	\$2,000	\$4,000	\$7,000

Variable	Minimum	Most Likely	Maximum
Hosting, tools, and setup	\$1,000	\$2,000	\$4,000

A triangular distribution was used for each variable because there is a most likely estimate, while lower and higher costs are still possible.

A total of 10,000 simulation trials were conducted.

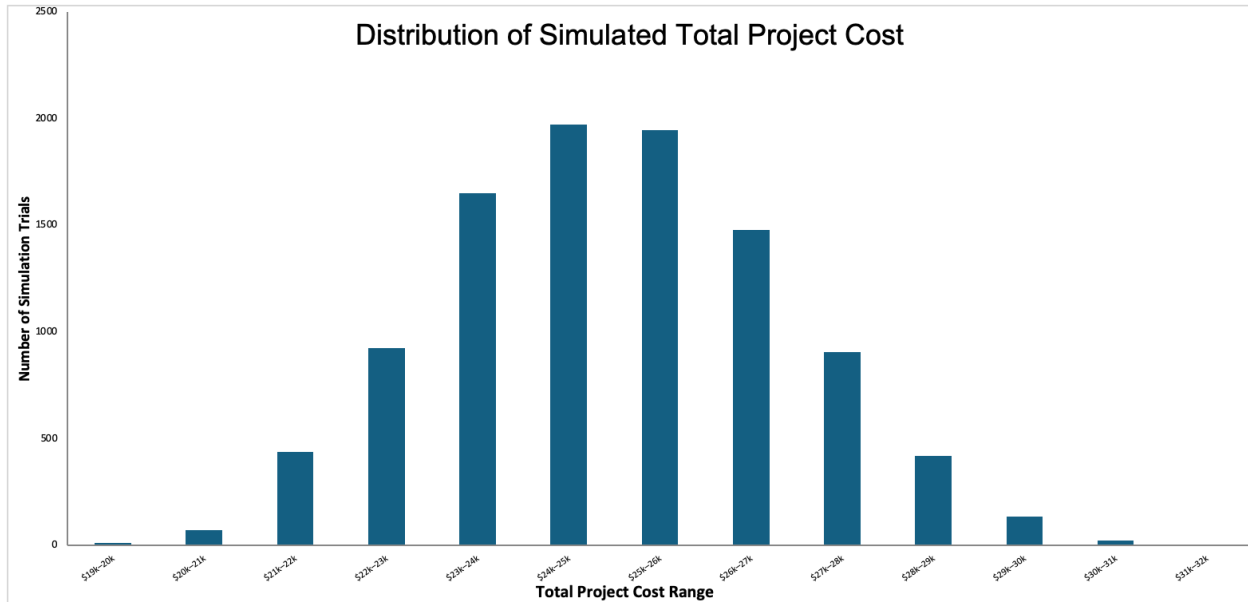
Simulation Results:

Result	Total Project Cost
Average cost	\$25,001
10th percentile	\$22,613
90th percentile	\$27,474
Probability of exceeding \$25,000	49.2%

The results show that the average project cost is almost exactly the original \$25,000 estimate. However, there is about a 49% chance that the project could exceed the \$25,000 budget. Therefore, this shows that using only one estimated cost for a project hides a significant amount of uncertainty. In addition, the simulation also shows that about 90% of the simulated project costs were below approximately \$27,474, while the lower-cost outcomes could fall closer to \$22,613.

Management Decision

Based on the simulation, management should not treat the original \$25,000 estimated cost as a guaranteed budget. Since there is almost a 50% chance of exceeding that amount. Moreover, it would be reasonable to include a small contingency reserve in the project budget. For example, management could budget closer to \$27,500 rather than exactly \$25,000. This would provide more room for unexpected testing, debugging, or development costs without immediately putting the project over budget.



Part K – Risk Mitigation and Monitoring

The three highest priority risks identified for the Expense Tracker were schedule delays, integration or configuration problems, and security or privacy issues. A mitigation plan, contingency plan, and an early warning signal are to be implemented so the team can respond before the problem becomes more serious.

Schedule Delay:

For mitigation, the team should keep the project scope realistic and divide the work into smaller tasks. The team should check progress regularly as well, and lower-priority features should not be added unless the main features are already working.

For a contingency plan, if the project begins to fall behind, then the team should postpone less important features and focus only on the core Expense Tracker functions needed for the final prototype.

Early warning signs would include missed task deadlines, unfinished work that carries over for several days, or testing taking much longer than expected.

Integration or Configuration Problems:

For mitigation, the team should test features as they are added instead of waiting until the end of the project. Changes should also be checked to make sure they don't break other parts of the application.

For a contingency plan, if an integration or configuration causes a major problem, the team should return to the last working version. Then the team should identify what changes caused the issue and fix that problem before continuing with other features.

Early warning signals would include failing automated tests or if a feature works by itself but causes a problem when connected with another part of the application.

Security or Privacy Issues:

For mitigation, the application should only collect the financial information it actually needs. Furthermore, access controls should be tested to make sure users can only view their own records.

For a contingency plan, if a privacy or security problem is discovered, the affected feature should be disabled until the issue is fixed. The team should also determine whether any user information was exposed or accessed incorrectly.

Early warning signs would include unexpected access errors appearing or testing showing the financial information not being properly protected.

Monitoring

The project manager should review these risks throughout the project instead of only checking them at the beginning or end. Furthermore, the team needs to update the risk scores whenever new information becomes available. Each new major risk should also be assigned to someone for the responsibility of watching it. In the case an early warning signal appears, the team should decide whether the normal mitigation is enough or whether the contingency plan needs to be used.

Vibe Coding Reflection

What Did Vibe Coding Teach Us?

Before Coding: We expected the basic transaction tracking features to be relatively easy because the project mainly involved storing and displaying financial records.

During Development: Some parts took longer than expected, especially adapting existing Django code, testing different features, and fixing problems between different parts of the application.

AI: AI was useful for generating starting code, tests, and helping debug errors. However, AI-generated code sometimes made incorrect assumptions about how our existing project worked. For example, some generated budget tests expected different data than the application actually returned, so we had to review and correct the tests ourselves.

Estimation: Our original estimates were somewhat optimistic because testing and debugging required more time than expected.

Risk: Technical problems did occur during development, including errors found through automated testing and configuration issues.

Scope: We focused on completing the core Expense Tracker features and left features such as savings goals, recurring transactions, receipt uploads, and bank integration for future development.

This showed us that building software is not only about writing code. Managing scope, testing, responding to problems, and deciding what not to build were also important parts of completing the project.